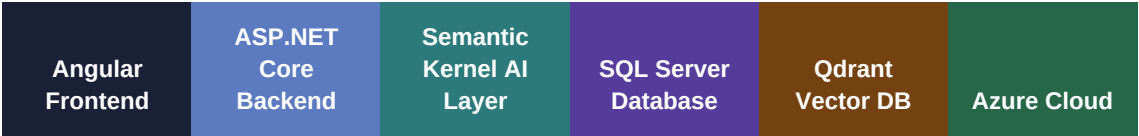




Tech Stack — Why We Chose Every Technology

Graduation Project | Full-Stack AI SaaS Platform

System Architecture at a Glance



The platform is split into 3 tiers: Frontend (Angular), Backend (ASP.NET Core + Semantic Kernel), and Storage (SQL Server + Qdrant). All tiers are deployed on Azure.

1. Frontend — Angular + ApexCharts + Bootstrap



What it does

Angular is the framework that builds every page the Owner and Manager see: the Login page, the Dashboard with charts, the Report Upload page, the Alerts page, and the AI Chat page.

Why Angular over React or Vue?

Reason	Explanation
TypeScript by default	Angular forces TypeScript. This means fewer runtime bugs because errors are caught at compile time.
Built-in structure	Angular provides Modules, Services, Guards, and Interceptors out of the box. For a 5-person team, this structure is invaluable.
Enterprise grade	InsightX is a B2B SaaS platform, not a simple webpage. Angular is designed for large applications.
Team familiarity	The team already has Angular experience from their coursework. Switching to React would have added unnecessary risk.

ApexCharts — Why not Chart.js or D3?

Reason	Explanation
Angular module ready	ApexCharts has an official ng-apexcharts Angular module. Chart.js requires manual DOM manipulation.
Rich chart types	Line, Bar, Area, Radial, Heatmap — all built-in and responsive by default.
Minimal code	A full chart with tooltip, zoom, and legend takes 10 lines. D3 would take 100+ lines for the same.

2. Backend — ASP.NET Core Web API

Business Logic + API Layer

What it does

ASP.NET Core handles all API endpoints: authentication, report management, alert generation, dashboard data, and AI agent calls. It also hosts the Semantic Kernel AI layer in the same process.

Why ASP.NET Core over Node.js or Django?

Reason	Explanation
Team skill set	All 5 team members are trained in .NET. Using Node or Django would require learning a new language and framework.
Semantic Kernel is native .NET	Semantic Kernel — the AI orchestration library we use — is built by Microsoft for C#. Using it in .NET is the most straightforward.
Performance	ASP.NET Core consistently ranks in the top 3 fastest web frameworks globally (TechEmpower benchmarks).
Security built-in	JWT authentication, role-based authorization, CORS, and HTTPS are all first-class citizens in ASP.NET Core.
Single deployable	Frontend + Backend + AI all deploy as one unit on Azure App Service. Node + Python would require separate services.

Entity Framework Core — Why not Dapper or raw SQL?

Reason	Explanation
Code-first migrations	Define tables in C# classes. EF generates and runs SQL migrations automatically. No need for manual SQL scripts.
LINQ queries	Write database queries in C# with full IntelliSense support. Type-safe — no string SQL bugs.
CompanyId filtering	Add a global query filter for CompanyId once in DbContext and every query automatically filters by it.

3. AI Layer — Semantic Kernel

AI Orchestration | LLM Integration | Agent Logic

What it does

Semantic Kernel is the Microsoft library that connects ASP.NET Core to Large Language Models (LLMs) like GPT-4. It handles prompt templates, chat history, function calling, and agent planning — all from C#.

Why Semantic Kernel over LangChain?

Reason	Explanation
Language match	LangChain is Python. Semantic Kernel is C#. Since our backend is ASP.NET Core, Semantic Kernel is the natural choice.
No Python microservice	LangChain would require a separate Python FastAPI server, HTTP calls between services, and managing two languages.
Microsoft support	Semantic Kernel is actively maintained by Microsoft and integrates natively with Azure OpenAI.
Simpler architecture	One codebase, one language, one deployment. LangChain adds architectural complexity through multiple services.

What Semantic Kernel handles in InsightX AI

- Document Normalization: send report text to LLM, get structured JSON back.
- Anomaly Alert Generation: send KPI data to LLM, get human-readable alert message.
- AI Agent Chat: manage conversation history, retrieve chunks, call LLM, return answer.
- Embedding: convert text chunks to vectors for storage in Qdrant.

4. Database — SQL Server

Structured Data Storage

What it stores

Table	Purpose
Companies + Users	Who is subscribed. Roles and department assignments.
KPIs + Thresholds	What each company considers a problem.
Reports log	Who uploaded what report and when.
Extracted Metrics	Structured numbers extracted from reports by AI.
Historical Metrics	Monthly KPI values used by anomaly detection.
Alerts	All alerts fired, with message and recommendation.
Conversations	Full history of AI Chat questions and answers.

Why SQL Server over PostgreSQL or MongoDB?

Reason	Explanation
EF Core integration	Entity Framework Core has the most mature, feature-rich support for SQL Server. All EF fe
Azure SQL	SQL Server deploys to Azure SQL Managed Instance in one click. No configuration. Autom
Team experience	The team has used SQL Server in previous coursework. PostgreSQL would be functionally
Why not MongoDB	InsightX data is highly relational (Companies → Users → Departments → Reports → KPIs)

5. Vector Database — Qdrant

Report Embeddings Storage | Semantic Search

What it does

Qdrant stores the vector embeddings of all text chunks extracted from company reports. When the Owner asks a question, Qdrant finds the most semantically similar chunks so the AI Agent can answer from real report data instead of making things up.

Why Qdrant over FAISS or ChromaDB?

Reason	Explanation
Production ready	FAISS is in-memory only — data is lost on restart. Qdrant persists data to disk and is designed for production.
Filtering support	Qdrant supports payload filtering natively. We filter every search by company_id to ensure results are only from the relevant company.
REST + gRPC API	Qdrant exposes a clean REST API that ASP.NET Core can call directly using HttpClient. No need for a separate API layer.
Azure Cloud ready	Qdrant Cloud has a managed version that deploys on Azure. ChromaDB has weaker cloud support.
.NET SDK	Qdrant provides an official .NET client library that integrates seamlessly with Semantic Kernel.

How data is stored in Qdrant

Field	Description
Collection	'reports' — one collection for all companies.
Vector	1536-dimension float array from OpenAI embedding model.
Payload	company_id, department_id, report_id, month, year, text.
Isolation	Every search query MUST include filter: company_id == X.

6. Deployment — Azure

Cloud Infrastructure

What deploys where

Service	Purpose
Azure App Service	Hosts the ASP.NET Core backend + Semantic Kernel.
Azure Static Web Apps	Hosts the Angular frontend. Free tier available.
Azure SQL	Managed SQL Server database. Automatic backups.
Azure OpenAI	LLM endpoint (GPT-4) used by Semantic Kernel.
Qdrant Cloud	Vector database managed service.

Why Azure over AWS or GCP?

Reason	Explanation
Microsoft ecosystem	ASP.NET Core + SQL Server + Semantic Kernel + Azure OpenAI are all Microsoft products.
Azure OpenAI	To use GPT-4 with Semantic Kernel in production, Azure OpenAI is the recommended endpoint.
Student credits	Microsoft Azure gives free credits to students via Azure for Students program. This allows for free deployment.
Single vendor	All infrastructure on one platform means one dashboard, one billing account, and one set of permissions.

Full Decision Table — We Use vs Alternatives

Technology	We Use	Alternatives	Why We Chose It
Frontend	Angular	React, Vue	Team skill + TypeScript + Enterprise structure
Charts	ApexCharts	Chart.js, D3	Angular module + rich types + minimal code
Backend	ASP.NET Core	Node.js, Django	Team skill + native Semantic Kernel + performance
ORM	EF Core	Dapper, raw SQL	Code-first migrations + global CompanyId filter
AI Orchestration	Semantic Kernel	LangChain, LlamaIndex	C# native + no Python service + Microsoft support
Relational DB	SQL Server	PostgreSQL, MongoDB	EF integration + Azure SQL + team experience
Vector DB	Qdrant	FAISS, ChromaDB, Pinecone	Persistent + filtering + .NET SDK + Azure ready
Cloud	Azure	AWS, GCP	Microsoft ecosystem + Azure OpenAI + student credits

Architecture Summary

The key architectural decision in InsightX AI is keeping everything in one language and one ecosystem. By choosing Angular + ASP.NET Core + Semantic Kernel + Azure, the team avoids the complexity of a Python microservice, reduces deployment surface, and leverages existing skills. The only exception is Qdrant which runs as a separate managed cloud service but is called via a simple REST API from C#.

The One Rule: Stay in the .NET Ecosystem

- Never add a Python service — Semantic Kernel handles all AI needs from C#.
- Never add a second backend — one ASP.NET Core process does everything.
- Never switch ORM — Entity Framework Core handles all SQL Server operations.
- Qdrant is the only external service — everything else is Azure-hosted .NET.